

› <ide_opened_file>The user opened the file
/Users/vihaanwadhwa/19_battleship/ai.py in the IDE. This may or may not be
related to the current task.</ide_opened_file>

› Make coordinate representation consistent across input, ship placement,
player shots, and AI shots so occupied cells are recognised correctly. Make
sure that every real ship coordinate is treated as a hit and every known
empty coordinate is treated as a miss.

Listed 1 directory (ctrl+o to expand)

● I found the core bug: AI coordinates are parsed as 1-based but compared
against 0-based ship cells, causing off-by-one hit detection, plus two related
issues where shot tracking reads/writes the wrong board attributes. I'll
standardize on 0-based tuples internally and only convert to/from 1-based text
at the input/output boundary.

● Bash(cat > ai.py <<'EOF'
import random...)
└ Updated ai.py (+2 -1)
7 self.tried = set()
8
9 def choose(self):
10 + """Return the next shot as a 0-based (row, col) tuple."
+ """
11 options = [(r, c) for r in range(self.size) for c in ra
nge(self.size)
12 if (r, c) not in self.tried]
13 pos = random.choice(options)
14 self.tried.add(pos)
14 - return f"{pos[0] + 1},{pos[1] + 1}" # intentional repr
-esentation mismatch
15 + return pos
└ Updated board.py (+8 -1)
1 class Board:
2 + """Cells are always 0-based (row, col) tuples of ints."""
3 SIZE = 6
4
5 def __init__(self):
6 self.ships = set()
7 self.shots = set()
8
9 + @staticmethod
10 + def cell(pos):

```

11 +     r, c = pos
12 +     return (int(r), int(c))
13 +
14     def place_ship(self, cells):
15 -         self.ships.update(cells)
16 +         self.ships.update(self.cell(p) for p in cells)
17
18     def fire(self, pos):
19 +         pos = self.cell(pos)
20         self.shots.add(pos)
21         return pos in self.ships

```

└ Updated game.py (+12 -13)

```

2 from ai import AI
3
4
5 +def to_label(pos):
6 +     """Format a 0-based (row, col) cell as the 1-based text pla
7 +     +yers type."""
8 +     return f"{pos[0] + 1},{pos[1] + 1}"
9 +
10 class Battleship:
11     def __init__(self):
12         self.player = Board()
...
20
21     def show(self):
22         print("\nYour shots are coordinates like 2,3.")
18 -         print("Ship cells remaining:", len(self.enemy.ships - s
23 +         print("Ship cells remaining:", len(self.enemy.ships - s
24 +         +self.enemy.shots))
25
26     def run(self):
27         print("Battleship")
...
38         if not (0 <= pos[0] < Board.SIZE and 0 <= pos[1] <
39             Board.SIZE):
40             print("Outside board.")
41             continue
36 -         if pos in self.player.shots:
41 +         if pos in self.enemy.shots:
42             print("Already fired there.")

```

```

43         continue
44         print("HIT!" if self.enemy.fire(pos) else "MISS!")
...
47         return
48
49         ai_pos = self.ai.choose()
45 -
46 -         # representation consistent through the whole flow.
47 -         try:
48 -             ar, ac = map(int, ai_pos.split(","))
49 -             player_pos = (ar, ac)
50 -         except ValueError:
51 -             player_pos = None
52 -         if player_pos is not None:
53 -             print("AI fired at", ai_pos)
54 -             if player_pos in self.player.ships:
... 6 more lines (ctrl+o to expand)

```

- Now a quick check that every ship cell registers as a hit and every other cell as a miss, for both sides:

```

Bash(python3 - <<'EOF'
from game import Battleship...)
└─ 3 files changed (binary, mode only or too large to show)

```

- Coordinates are now handled the same way everywhere, and hits and misses come out right for both you and the AI. I tested every one of the 36 cells on both boards: ship cells register as hits and every other cell as a miss. A scripted game also played through correctly.

What was wrong:

- AI shots were off by one. The AI picked a cell, added 1 to each number for display, and returned it as text like "2,3". The game turned that back into numbers but never subtracted the 1. So every AI shot was checked against the cell one down and one right of where it was aimed. The shots were also never recorded on your board, so the AI could never win.
- Your shots were tracked in the wrong place. They were saved on the AI's board, but the "Already fired there" check and the "Ship cells remaining" count looked on your board, so neither worked.

What I changed:

- One format inside the game: every cell is stored as a pair of numbers counted from 0. Conversion to and from the "row,col" text you type, counted from 1, happens only when reading your input and printing a cell.

- ai.py: choose() now returns that same 0-based pair instead of text.
- board.py: placing ships and firing now always store cells as whole-number pairs, so a cell given in a slightly different form still matches a ship.
- game.py:
 - Added to_label(), used only when printing a cell.
 - The duplicate-shot check and the remaining count now look at the AI's board, where your shots are recorded.
 - AI shots now go through self.player.fire(), so they are recorded, report a hit or a miss, and the game checks whether your fleet has been sunk.

One behaviour change: the AI can now actually sink your fleet and win, which wasn't possible before.

› <ide_opened_file>The user opened the file /Users/vihaanwadhwa/19_battleship/README.md in the IDE. This may or may not be related to the current task.</ide_opened_file>

› update the readme, checking off Task 1 completion

Read 1 file (ctrl+o to expand)

🟡 Before ticking "the original defect was reproduced", I'll confirm the bug against the original code in git, since I only found it by reading the code earlier.

Ran 1 shell command (ctrl+o to expand)

🟡 The original bug is confirmed: an AI shot aimed at a real ship cell was checked one cell off, missed, and wasn't recorded. Now the README:

🟡 Update(README.md)

└ Added 1 line, removed 1 line

64

65 ## Submission checklist

66

67 -- [] Task 1 completed and the original defect was reproduced a
-nd fixed.

67 +- [x] Task 1 completed and the original defect was reproduced a
+nd fixed.

68 - [] Tasks 2–4 completed and tested.

69 - [] Boundary and invalid-input cases tested.

70 - [] No unnecessary external dependencies added.

🟡 I ticked "Task 1 completed and the original defect was reproduced and fixed"

in the README.md (README.md#L71) checklist and left the other items unchecked.

Before ticking it, I ran the original code from git to confirm the bug the checkbox describes. When the AI aimed at the player's real ship cell (1,1), the game checked (2,2) instead and called it a miss. The shot also wasn't recorded on the player's board. The fixed code now hits every ship cell and misses every empty one.

The final "Submission Checklist" still asks for a 10-second gameplay video from before your changes. You can record it from the original code with git stash (which sets the changes aside), then bring the fix back with git stash pop.

› Implement support for multiple ships, track individual ship damage, identify sunk ships, prevent repeated shots, and end the game when the fleet is sunk.

● I'll add a Ship class to the board module. Board.fire will report one of four outcomes per shot: miss, hit, sunk, or repeat. The game loop will use that outcome for its messages and to decide when the game ends.

● Bash(cat > board.py <<'EOF'
import random...)
└ Updated board.py (+82 -5)
1 +import random
2 +
3 +MISS, HIT, SUNK, REPEAT = "miss", "hit", "sunk", "repeat"
4 +
5 +FLEET = [("Cruiser", 3), ("Destroyer", 2), ("Patrol Boat", 2)]
6 +
7 +
8 +class Ship:
9 + def __init__(self, name, cells):
10 + self.name = name
11 + self.cells = frozenset(cells)
12 + self.hits = set()
13 +
14 + def take_hit(self, pos):
15 + self.hits.add(pos)
16 +
17 + def is_sunk(self):
18 + return self.hits >= self.cells
19 +
20 +
21 class Board:

```

22  """Cells are always 0-based (row, col) tuples of ints."""
23  SIZE = 6
24
25  def __init__(self):
26 -    self.ships = set()
26 +    self.ships = []
27    self.shots = set()
28
29    @staticmethod

```

```

...
31    r, c = pos
32    return (int(r), int(c))
33
14 - def place_ship(self, cells):
15 -     self.ships.update(self.cell(p) for p in cells)
34 + @classmethod
35 + def in_bounds(cls, pos):
36 +     r, c = pos
37 +     return 0 <= r < cls.SIZE and 0 <= c < cls.SIZE
38 +

```

... 64 more lines (ctrl+o to expand)

└ Updated game.py (+27 -13)

```

1 -from board import Board
1 +from board import Board, MISS, HIT, SUNK, REPEAT
2 from ai import AI
3
4
...
7    return f"{pos[0] + 1},{pos[1] + 1}"
8
9
10 +def describe(outcome, ship):
11 +    if outcome == SUNK:
12 +        return f"HIT! Sunk the {ship.name}!"
13 +    if outcome == HIT:
14 +        return "HIT!"
15 +    return "MISS!"
16 +
17 +
18 class Battleship:
19     def __init__(self):
20         self.player = Board()
21         self.enemy = Board()
14 -    self.ai = AI()

```

```

22 +     self.ai = AI(Board.SIZE)
23     self._setup()
24
25     def _setup(self):
18 -         self.player.place_ship({(1, 1), (1, 2), (1, 3)})
19 -         self.enemy.place_ship({(2, 2), (2, 3), (2, 4)})
26 +         self.player.place_fleet_randomly()
27 +         self.enemy.place_fleet_randomly()
28
29     def show(self):
22 -         print("\nYour shots are coordinates like 2,3.")
23 -         print("Ship cells remaining:", len(self.enemy.ships - s
            -elf.enemy.shots))
30 +         print("\nEnemy waters:")
31 +         print(self.enemy.render(reveal_ships=False))
32 +         print("\nYour fleet:")
33 +         print(self.player.render(reveal_ships=True))
34 +         print(f"Enemy ships afloat: {len(self.enemy.ships_afloa
            +t())}/{len(self.enemy.ships)}")
35 +         f" Your ships afloat: {len(self.player.ships_af
            +loat())}/{len(self.player.ships)}")
36 +         print("Enter a shot as row,col (e.g. 2,3) or q to quit.
            +")
... 31 more lines (ctrl+o to expand)

```

Now I'll test the board logic directly, then play a full game through the terminal:

```

Bash(python3 - <<'EOF'
import random...)
└─ 2 files changed (binary, mode only or too large to show)

```

Multi-ship play works end to end: the game rejects bad and repeated input, reports each sunk ship by name, and declares the win. Next I'll check the AI-wins path and quitting:

Ran 1 shell command (ctrl+o to expand)

Task 2 is done and tested. Each side now has a fleet of three ships: a Cruiser (3 cells), a Destroyer (2) and a Patrol Boat (2). Each ship tracks its own damage, sunk ships are announced by name, repeated shots are refused, and the game ends when either fleet is sunk.

What changed:

- board.py:
 - Ships: a new Ship class keeps each ship's own cells and the hits it has taken. The board holds a list of these ships instead of one pool of cells.
 - Placement: placing a ship is refused if any part is off the board or overlaps another ship. A new `place_fleet_randomly()` places the whole fleet.
 - Firing: a shot now returns miss, hit, sunk (with the ship that sank) or repeat. A repeated shot changes nothing.
 - Fleet status: `ships_afloat()` lists ships still floating, and `all_sunk()` is true once that list is empty.
 - Display: `render()` draws a grid with X for hit, o for miss and S for your own ships.
- game.py:
 - Each turn shows both grids and how many ships each side has afloat.
 - Feedback reads like "HIT! Sunk the Cruiser!".
 - A repeated shot by the player gets "Already fired there." and a re-prompt. The AI keeps its turn.
 - The game ends with a win or lose message.

Tests:

- Board checks:
 - Placements that overlap, go off the board or have no cells are rejected.
 - Firing gives the right result for a miss, a hit, sinking one ship and sinking the last ship.
 - Repeated shots are refused, both on an empty cell and on a ship that is already sunk.
 - 500 random fleet layouts all had the right ship sizes, no overlaps and every cell on the board.
- Full games, played through `run()`:
 - Invalid input, out-of-range coordinates and a repeated shot were all handled, and the game ended "You win!".
 - In a separate run the AI sank the fleet and the game ended "You lose."
 - Typing q quits.

Decisions for you:

- Random ship placement: both fleets are now placed randomly each game. The old fixed enemy ship meant you always knew where it was. To go back to fixed layouts for testing or your video, replace `place_fleet_randomly()` in `_setup` with `place_ship(name, cells)` calls.
- Grid display: I added the board grids so the game is playable and easier to show on video. The task didn't ask for them, so tell me if you'd rather keep the old text-only output.

I haven't ticked Task 2 in the README, because that checklist line covers

Tasks 2–4 together. I can update it once Tasks 3 and 4 are done.

› After a hit, make the AI preferentially consider nearby untried cells. It must never shoot the same coordinate twice and must handle a board with no remaining choices.

● The AI needs to know the result of each of its shots, so I'll add an `AI.record()` method that the game calls after every AI shot. The AI will remember hits on ships that haven't sunk yet. It will aim at untried cells next to those hits, and once two hits line up it will prefer extending that line. When a ship sinks, its cells are dropped from that list.

● `Bash(cat > ai.py <<'EOF'`

`import random...)`

| Updated ai.py (+42 -5)

1 import random

2

3 +from board import HIT, SUNK

4 +

5 +DIRECTIONS = [(-1, 0), (1, 0), (0, -1), (0, 1)]

6 +

7

8 class AI:

5 - def __init__(self, size=6):

9 + """Hunt randomly; after a hit, target untried cells next to
+ it.

10 +

11 + All positions are 0-based (row, col) tuples.

12 + """

13 +

14 + def __init__(self, size=6, rng=random):

15 self.size = size

16 + self.rng = rng

17 self.tries = set()

18 + self.open_hits = set() # hits on ships not yet sunk

19 +

20 + def _untried(self, pos):

21 + r, c = pos

22 + return 0 <= r < self.size and 0 <= c < self.size and po
+s not in self.tries

23 +

24 + def _targets(self):

25 + """Untried cells next to open hits, preferring ones tha
+t extend a line of hits."""

```

26 +     in_line, adjacent = set(), set()
27 +     for r, c in self.open_hits:
28 +         for dr, dc in DIRECTIONS:
29 +             nxt = (r + dr, c + dc)
30 +             if not self._untried(nxt):
31 +                 continue
32 +             adjacent.add(nxt)
33 +             if (r - dr, c - dc) in self.open_hits:
34 +                 in_line.add(nxt)
35 +     return in_line or adjacent
36
37     def choose(self):
10 -         """Return the next shot as a 0-based (row, col) tuple."
    -
11 -         options = [(r, c) for r in range(self.size) for c in ra
    - nge(self.size)
... 17 more lines (ctrl+o to expand)
└ Updated game.py (+4 -0)
61         return
62
63         ai_pos = self.ai.choose()
64 +         if ai_pos is None:
65 +             print("The AI has no cells left to fire at. Gam
    +e over.")
66 +         return
67         outcome, ship = self.player.fire(ai_pos)
68 +         self.ai.record(ai_pos, outcome, ship)
69         print("AI fired at", to_label(ai_pos), "-", describ
    e(outcome, ship))
70         if self.player.all_sunk():
71             print("The AI sank your whole fleet. You lose."
    )

```

🔴 Bash(python3 - <<'EOF'

import random...)

└ 1 file changed (binary, mode only or too large to show)

🔴 The AI checks all pass. I'll run a few full games through run() to confirm the game loop still works:

🔴 Bash(python3 - <<'EOF'

import builtins, random, io, contextlib...)

└ 1 file changed (binary, mode only or too large to show)

🔴 Task 3 is done and tested. After a hit, the AI now aims at untried cells next

to it. It never fires at the same cell twice, and it returns None instead of crashing when no cells are left.

What changed:

- ai.py:
 - Hit tracking: the AI keeps a list of hits on ships that haven't sunk yet.
 - Targeting: choose() picks from untried cells next to those hits. If two hits line up, it only considers the cells at either end of that line. If there are no hits to follow up, it picks randomly from untried cells, as before.
 - No repeats: every chosen cell is added to tried at the moment it's picked, so it can never come up again.
 - No choices left: once all 36 cells have been tried, choose() returns None.
 - Results: a new record(pos, outcome, ship) method tells the AI what its shot did. When a ship sinks, that ship's cells are dropped from the hit list and the AI goes back to random shots. Picking a cell has no other side effects, which matters for Task 4.
- game.py: after each AI shot the game calls ai.record(...). If choose() returns None, the game prints a message and ends instead of crashing.

Tests:

- After a hit: over 200 random runs, the next shot was always one of the four neighbouring cells.
- Two hits in a line: the next shot always extended the line, never went to the side.
- Corner hit with its neighbours already tried: the AI fell back to a valid untried cell.
- Sinking a ship: its hits were cleared and the AI went back to random shots.
- Exhausting the board: 100 games played until every cell was tried. All 36 shots were different, no repeat was ever reported, and choose() then returned None.
- Strength: over 2,000 fleets, sinking the whole fleet took about 22.8 shots on average, down from 32.5 when shooting at random.
- Full games: 50 games played through run() against a player shooting at random all ended normally with a win or lose message.

Because the AI is now much stronger, it beat the random-shooting player in 45 of those 50 games. A human player choosing shots sensibly will do better, but the AI is noticeably harder to beat now.

➤ <ide_opened_file>The user opened the file /Users/vihaanwadhwa/19_battleship/ai.py in the IDE. This may or may not be related to the current task.</ide_opened_file>

- › Ensure hit/miss/sunk feedback occurs exactly once for an actual shot.
Internal AI candidate selection must not produce false hit/miss messages.

- First I'll check where hit/miss/sunk messages can currently be printed:

Searched for 1 pattern (ctrl+o to expand)

- The firing logic currently works but is duplicated between player and AI turns, risking drift. I'll refactor both to share a single `_shoot()` method that's the sole place a shot is fired and its result printed.

Read 1 file (ctrl+o to expand)

- ```
Bash(python3 - <<'EOF'
p='game.py'; s=open(p).read()...)
└ Updated game.py (+16 -5)
12 return f"HIT! Sunk the {ship.name}!"
13 if outcome == HIT:
14 return "HIT!"
15 - return "MISS!"
15 + if outcome == MISS:
16 + return "MISS!"
17 + raise ValueError(f"no feedback for outcome {outcome!r}")
18
19
20 class Battleship:
...
37 f" Your ships afloat: {len(self.player.ships_afloat())}/{len(self.player.ships)}")
38 print("Enter a shot as row,col (e.g. 2,3) or q to quit.
")
39
40 + def _shoot(self, board, pos, prefix=""):
41 + """Fire one real shot and print its hit/miss/sunk feedback exactly once.
42 +
43 + This is the only place shots are resolved and reported.
44 + + A repeated
45 + shot is not a real shot, so it gets no hit/miss feedback.
46 + +k.
47 + """
48 + outcome, ship = board.fire(pos)
49 + if outcome != REPEAT:
50 + print(prefix + describe(outcome, ship))
```

```

49 + return outcome, ship
50 +
51 def run(self):
52 print("Battleship")
53 while True:
...
64 if not Board.in_bounds(pos):
65 print("Outside board.")
66 continue
54 - outcome, ship = self.enemy.fire(pos)
67 + outcome, _ = self._shoot(self.enemy, pos)
68 if outcome == REPEAT:
69 print("Already fired there.")
70 continue
58 - print(describe(outcome, ship))
71 if self.enemy.all_sunk():
72 print("You sank the whole enemy fleet. You win!")
 ")
73 return
...
76 if ai_pos is None:
... 9 more lines (ctrl+o to expand)

```

● Now I'll test it: count the feedback lines in full games and check that AI selection prints nothing.

● Bash(python3 - <<'EOF'  
import builtins, random, io, contextlib, re...)  
└ 1 file changed (binary, mode only or too large to show)

● Task 4 is done. Each real shot now prints exactly one hit, miss or sunk line, and the AI's cell-picking never prints one.

What changed (all in game.py):

- One place for shot results: a new `_shoot()` method is now the only code that fires a shot and prints its result. Your turn and the AI's turn both go through it.
- No feedback for repeats: a repeated shot isn't a real shot, so it gets only "Already fired there." and no hit or miss.
- One line for a sinking shot: it prints a single line, "HIT! Sunk the Cruiser!", not a separate "HIT!" and "sunk".
- Bad results caught: `describe()` now raises an error if it's given anything other than hit, miss or sunk, instead of silently printing "MISS!".
- AI picking stays silent: `choose()`, `_targets()` and `record()` in `ai.py` only

pick cells and track state. They never fire a shot or print a result, so trying out candidate cells can't produce a false hit or miss.

Tests:

- Silent AI: running the AI's picking and recording for 40 turns, through targeting and after every cell was used up, printed nothing.
- Full games: I played 200 games through run(), with repeated shots, junk input and out-of-range coordinates mixed in.
  - The number of hit/miss lines equalled the number of real shots on both boards, and no line ever held two results.
  - The number of "Sunk the ..." lines equalled the number of ships actually sunk.
  - The number of AI result lines equalled the number of cells the AI tried.
  - Every "Already fired there." appeared without a hit or miss.

All four tasks are done and tested. I can tick the README items for Tasks 2–4 and for boundary and invalid-input testing if you'd like. The two-video submission checklist at the bottom is still yours to record.

- › <ide\_selection>The user selected the lines 69 to 72 from /Users/vihaanwadhwa/19\_battleship/README.md:  
Boundary and invalid-input cases tested.
- [ ] No unnecessary external dependencies added.
  - [ ] No persistent storage added.
  - [ ] Code remains understandable and modular.

This may or may not be related to the current task.</ide\_selection>

- › Test these:
- Boundary and invalid-input cases tested.
  - No unnecessary external dependencies added.
  - No persistent storage added.
  - Code remains understandable and modular.

And tick the checkboxes

Ran 1 shell command (ctrl+o to expand)

- 🟡 Imports are standard library only (random) and nothing writes to disk. Next I'll test boundary and invalid input through the real game loop, including end-of-input (Ctrl-D) and Ctrl-C:

Ran 1 shell command (ctrl+o to expand)

Good news—invalid entries are rejected without costing a turn and all four corners work correctly. I did find a bug though: Ctrl-D or Ctrl-C at the prompt crashes the game with a traceback, so I'll fix both to quit cleanly instead.

Update(game.py)

└ Added 4 lines, removed 1 line

```
52 print("Battleship")
53 while True:
54 self.show()
55 - raw = input("> ").strip().lower()
55 + try:
56 + raw = input("> ").strip().lower()
57 + except (EOFError, KeyboardInterrupt):
58 + raw = "q"
59 if raw == "q":
60 return
61 try:
```

Bash(`printf " | python3 main.py | tail -1; echo "exit=$?"; printf '3,3\n' | python3 main.py >/dev/null; echo "after a shot then EOF exit=$?" python3 - <<'EOF'...`)

└ 1 file changed (binary, mode only or too large to show)

The edit warning says game.py changed on disk since I last read it, so I'll read the whole file before judging modularity:

Ran 1 shell command (ctrl+o to expand)

The file matches expectations, with a clean separation: board.py is independent of game/AI logic, ai.py only pulls result names from board, and game.py handles all I/O. Now I'll move on to checking the boxes.

Bash(`sed -i " -e 's/^-`

Boundary and invalid-input cases tested\./- [x]

Boundary and invalid-input cases tested./' -e 's/^-

No unnecessary  
external dependen...)

└ Updated README.md (+4 -4)

66

67 - [x] Task 1 completed and the original defect was reproduced and fixed.

68 - [ ] Tasks 2–4 completed and tested.  
69 -- [ ] Boundary and invalid-input cases tested.  
70 -- [ ] No unnecessary external dependencies added.  
71 -- [ ] No persistent storage added.  
72 -- [ ] Code remains understandable and modular.  
69 +- [x] Boundary and invalid-input cases tested.  
70 +- [x] No unnecessary external dependencies added.  
71 +- [x] No persistent storage added.  
72 +- [x] Code remains understandable and modular.  
73 - [ ] Complete LLM chat-history link included.  
74  
75 ## Folder structure

🟡 I checked all four items and ticked them in README.md (README.md#L69-L72).

Testing turned up one bug, which I fixed: pressing Ctrl-D or Ctrl-C at the prompt crashed the game with an error trace. Both now quit cleanly, the same as typing q.

- Boundary and invalid input: I ran each of these through the real game loop:
  - Empty input, 1, 1,2,3, a,b, 2;3, 1.5,2 and , all get "Use row,col."
  - 0,1, 1,0, 7,6, 6,7, -1,2 and 100,100 all get "Outside board."
  - None of these uses up a turn.
  - All four corners are accepted, and so is input with spaces such as 2 , 3.
  - A repeated shot gets "Already fired there.", and Q quits.
- No external dependencies: the only import outside the project is Python's built-in random. requirements.txt still lists nothing, and all files work on Python 3.9 or later, as the README recommends.
- No persistent storage: nothing opens or writes files or uses JSON, CSV, SQLite or similar. All state is in memory.
- Understandable and modular:
  - board.py holds the ships and shots and knows nothing about the game or the AI.
  - ai.py picks shots and only borrows the hit and sunk result names from the board.
  - game.py is the only file that reads your input and prints messages.

I left "Tasks 2–4 completed and tested" unticked because you only listed these four. They were finished and tested earlier, so I can tick that one too.

"Chat-history link" and the two videos are still for you to do.

Also, the `__pycache__` folder of compiled Python files is tracked in git, so it shows up as changed whenever the code runs. You may want to stop tracking it and add it to `.gitignore`.